

Practical 1

```
In [17]: import numpy as np

# Grid World parameters
GRID_SIZE = 4
GAMMA = 1.0          # Discount factor
THETA = 1e-4         # Convergence threshold
ACTIONS = ['U', 'D', 'L', 'R']

# Initialize value function
V = np.zeros((GRID_SIZE, GRID_SIZE))

# Terminal states
terminal_states = [(0, 0), (3, 3)]

# Policy: uniform random
policy_prob = 1 / len(ACTIONS)

def next_state_reward(state, action):
    """Returns next state and reward"""
    i, j = state

    if state in terminal_states:
        return state, 0

    if action == 'U':
        next_state = (max(i - 1, 0), j)
    elif action == 'D':
        next_state = (min(i + 1, GRID_SIZE - 1), j)
    elif action == 'L':
        next_state = (i, max(j - 1, 0))
    elif action == 'R':
        next_state = (i, min(j + 1, GRID_SIZE - 1))

    reward = -1
    return next_state, reward

# Policy Evaluation
while True:
    delta = 0
    new_V = V.copy()

    for i in range(GRID_SIZE):
        for j in range(GRID_SIZE):
            state = (i, j)

            if state in terminal_states:
                continue

            value = 0
            for action in ACTIONS:
                (ni, nj), reward = next_state_reward(state, action)
                value += policy_prob * (reward + GAMMA * V[ni][nj])

            new_V[i][j] = value
```

```
        delta = max(delta, abs(V[i][j] - new_V[i][j]))

V = new_V

if delta < THETA:
    break

print("State Value Function:")
print(V)
```

State Value Function:

```
[[ 0.          -13.99893866 -19.99842728 -21.99824003]
 [-13.99893866 -17.99861452 -19.9984378  -19.99842728]
 [-19.99842728 -19.9984378  -17.99861452 -13.99893866]
 [-21.99824003 -19.99842728 -13.99893866  0.          ]]
```